



CD SECURITY

For the Few Who Demand Perfection

AUDIT REPORT

Aidea

July 2026

Prepared by
radev_eth
ZanyBonzy
Pelz

Introduction

A time-boxed security review of the **Aidea** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A code review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About Aidea

Aidea is a launchpad template pairing an ERC20 token with an auto-evolving ERC721 identity NFT on Base. A user's first buy from the Uniswap V2 pair auto-mints a one-per-wallet NFT whose six on-chain traits (Background, Sigil, Weapon, Character, Border, Effect) evolve with trading behavior based on their entry time, holding duration, balance growth, etc. Holders who reach the final Character trait are eligible to claim a one-time, balance-doubling reward.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

Scope

The following folders and every file in them were in scope of the audit:

- `Aidea/tree/main/template_contracts.nft.sol`
- `Aidea/tree/main/template_contracts.token.sol`
- `Aidea/tree/main/template_contracts.traits.sol`

The following number of issues were found, categorized by their severity:

- Critical & High: 7 issues
- Medium: 10 issues
- Low and Informational: 13 issues

Findings Summary

ID	Title	Severity	Status
[C-01]	<code>hasClaimedReward</code> is not updated by <code>transferByNFTContract</code> , enabling repeated reward-pot drains via NFT hopping	Critical	Fixed
[C-02]	Direct transfers are misclassified as sells, potentially destroying earned traits and reward eligibility	Critical	Partially fixed
[C-03]	Never-traded accounts are treated as holding since the Unix epoch, allowing an immediate drain of the reward pot	Critical	Fixed
[C-04]	Zero-value <code>transferFrom</code> requires no allowance and lets anyone rewrite any user <code>UserStats</code>	Critical	Fixed
[H-01]	Buying through an aggregator credits the router as the buyer, so the real purchaser never receives an identity	High	Fixed
[H-02]	Any ERC-721 operator approval is an unbounded, unrevocable-in-effect claim on the owner's entire token balance	High	Acknowledged
[H-03]	The value advertised to NFT buyers is stale by construction and cannot be corrected	High	Fixed
[M-01]	<code>_safeMint</code> in the ERC20 transfer path makes buys revert for every contract recipient	Medium	Fixed
[M-02]	First buy after receiving tokens counts the pre-buy balance as extra leading to unearned WEAPON3 / CHARACTER3	Medium	Fixed
[M-03]	Missing EIP-4906 integration for auto-evolving metadata property	Medium	Fixed
[M-04]	<code>isCharacter2</code> anchors its 24-hour window to the global launch time instead of the user's first buy, making CHARACTER2 unreachable for anyone who buys after launch day	Medium	Fixed
[M-05]	Reward size is taken from the instantaneous balance and can be borrowed	Medium	Acknowledged

ID	Title	Severity	Status
[M-06]	The rarest Weapon and Character traits have no absolute balance floor and cost 2 wei	Medium	Fixed
[M-07]	The mint callback fires before <code>firstBuyStamp</code> is written, letting an attacker permanently brick any address's NFT for free	Medium	Fixed
[M-08]	The reward claim reverts instead of paying the remainder, permanently stranding the pot's tail	Medium	Fixed
[M-09]	Providing liquidity is booked as a sell and withdrawing it mints an identity to someone who never bought	Medium	Acknowledged
[M-10]	An identity can be force-minted onto any address, blocking every NFT transfer to it	Medium	Fixed
[L-01]	Every Border tier is 10× stricter than documented and contradicts the same percentage elsewhere in the file	Low	Fixed
[L-02]	NFT transfer overwrites the recipient's own <code>UserStats</code>	Low	Fixed
[L-03]	<code>setServerBaseURI</code> is gated on <code>tx.origin</code>	Low	Fixed
[L-04]	<code>speedMultiplier</code> tightens some traits while loosening others and the Sigil ignores it entirely	Low	Fixed
[L-05]	The Effect trait reads a streak rather than a sell count and is erased by a 1 wei buy	Low	Acknowledged
[L-06]	Splitting a position across wallets multiplies the reward payout	Low	Acknowledged
[L-07]	Reward pot is not funded at deployment	Low	Fixed
[L-08]	Metadata quotes use an unvalidated fixed router	Low	Acknowledged
[L-09]	Similar transaction fails due to overflow after a user's 255 consecutive action in the same direction	Low	Fixed
[L-10]	A sold-out NFT holder (ERC20 balance 0) reverts on-chain metadata via <code>quoteTokensInWETH(0)</code>	Low	Fixed
[L-11]	Re-calling <code>setUniswapV2Pair</code> resets <code>launchTimeStamp</code> , causing an underflow revert that bricks metadata for all prior buyers	Low	Fixed
[L-12]	<code>getDaysHeld</code> off-by-one: a same-block buy+sell (<code>lastSellStamp == lastBuyStamp</code>) is ignored, inflating days-held	Low	Fixed
[I-01]	"ETH Amount" metadata attribute is emitted in raw wei while "Token Amount" is human-scaled	Informational	Fixed

Detailed Findings

[C-01] `hasClaimedReward` is not updated by `transferByNFTContract`, enabling repeated reward-pot drains via NFT hopping

Severity

Impact:

High

Likelihood:

High

Description

`transferByNFTContract` moves the sender's entire ERC20 balance and copies the full stats struct (`userStats[to] = userStats[from]`), but never updates `hasClaimedReward`, which is mapped per-address. `claimReward` gates solely on `hasClaimedReward[msg.sender]` and recomputes eligibility from the copied stats/balance. So a claimant can transfer their NFT (which carries balance + 90-day-old stats) to a fresh wallet — the NFT-transfer guard only checks `balanceOf(to) == 0` for the NFT — and claim again from the unflagged address, repeating across unlimited addresses. This "technically" defeats the "each user can claim once" invariant.

```
function transferByNFTContract(address from, address to) external {
    require(msg.sender == address(nftContract), "...");
    uint transferAmount = balanceOf(from);
    if (transferAmount > 0 && balances[from] > 0) {
        balances[from] = 0;
        balances[to] += transferAmount;
        emit Transfer(from, to, transferAmount);
    }
    userStats[to] = userStats[from]; // stats migrate...
    delete userStats[from];
    // ...but hasClaimedReward does NOT migrate
}
```

Recommendations

Update the claim flag with the position.

[C-02] Direct transfers are misclassified as sells,

potentially destroying earned traits and reward eligibility

Severity

Impact:

High

Likelihood:

High

Description

In `triggerUpdate`, any transfer where neither party is `uniswapV2Pair` falls into the else branch and still calls `updateUserStatsOnSell(from, value)`, which sets `lastSellStamp = block.timestamp`, `sellStreak += 1`, and `buyStreak = 0`. A holder who merely sends tokens to another without trading is treated as a seller. As a result:

- `getDaysHeld` now returns `0` (because `lastSellStamp > lastBuyStamp`), collapsing `computeSigilValue` to `NONE`.
- `isCharacter4`/`isCharacter5` require `lastSellStamp == 0` and are permanently disqualified.
- `isCharacter6` / `claimReward` eligibility is lost — a holder who qualified before sending 1 token can no longer claim.
- On the receiving side, `userStats[to].lastTokenAmount = balances[to]` while `firstBuyAmount` is untouched, so a large incoming transfer inflates the growth ratio and can fabricate `WEAPON3`/`CHARACTER3` without any buy.

```
function triggerUpdate(address from, address to, uint value) internal {
    if (from == uniswapV2Pair) {
        updateUserStatsOnBuy(to, uint128(value));
    } else {
        if (to == uniswapV2Pair) {
            updateUserStatsOnSell(from, uint128(value));
        } else {
            updateUserStatsOnSell(from, uint128(value)); // line 178:
plain transfer == sell
            userStats[to].lastTokenAmount = uint128(balances[to]);
        }
    }
    nftContract.triggerMetadataUpdate();
}
```

Recommendations

Only classify a transfer as a sell when `to == uniswapV2Pair`. For non-pair→non-pair transfers, do not call `updateUserStatsOnSell`. May just update `lastTokenAmount` snapshots

CD Security team:

Sender-side classification accepted as design and receiving side untouched - an inbound transfer alone still yields WEAPON3.

[C-03] Never-traded accounts are treated as holding since the Unix epoch, allowing an immediate drain of the reward pot

Severity

Impact:

High

Likelihood:

High

Description

`token.sol#claimReward()` gates the payout on `traits.sol#isCharacter6()`, which requires the caller to have held at least 0.1% of supply for 90 days. Holding time comes from `token.sol#getDaysHeld()`:

```
if (lastSellStamp == 0) {
    return fromDurationToDays(uint128(block.timestamp) -
    uint128(firstBuyStamp));
}
```

For an address that has never bought from the pair, `firstBuyStamp` is 0. The subtraction then measures from the Unix epoch rather than from a real purchase and the function returns 19,675 days.

`UserStats` is only populated on the buy path. Receiving tokens through a plain transfer runs the else-branch of `token.sol#triggerUpdate()`, which writes `userStats[to].lastTokenAmount` and nothing else - `firstBuyStamp`, `lastBuyStamp` and `lastSellStamp` all stay zero. `claimReward()` also never checks that the caller owns an identity NFT and it reads the live balance rather than `lastTokenAmount`.

So any freshly created address that is handed 0.1% of supply passes every check on its first transaction. Payout is $\min(\text{balance}, \text{totalSupply} / 400) = 25,000$ tokens and the attacker's capital is fully recoverable afterwards.

The second test starts the attacker with 25,000 tokens of working capital, cycles them through 40 fresh addresses (fund, claim, sweep - about 120 transactions in total) and ends holding the entire 1,000,000-token reward pot plus the original capital. No holding time and no prior interaction with the protocol are required at any point.

Recommendations

Reject accounts with no recorded purchase before computing a duration and derive the claim from tracked state rather than the spot balance:

```
function claimReward() public {
    require(!hasClaimedReward[msg.sender], "You have already claimed your reward");
    UserStats memory stats = userStats[msg.sender];
    require(stats.firstBuyStamp != 0, "no recorded purchase");
    ...
}
```

Add the same `firstBuyStamp != 0` guard inside `getDaysHeld()` and `getDaysHeldInHours()` so every downstream trait consumer is covered, returning `0` when it is unset.

Also restore the check the comment already promises. The line above the function reads *"If user's nft has the final character trait, they can claim from reward pot"*, but the body never touches the NFT contract - no `balanceOf`, no `ownerOf`, no `userToTokenId`. Compare `nft.sol#generateNFTTraits()`, which does gate on ownership. Requiring the claimant to hold their identity costs one call and closes a residual state in which an address ends up with a populated `firstBuyStamp` but no NFT:

```
require(nftContract.balanceOf(msg.sender) > 0, "no identity");
```

[C-04] Zero-value `transferFrom` requires no allowance and lets anyone rewrite any user `UserStats`

Severity

Impact:

High

Likelihood:

High

Description

`token.sol#transferFrom()` validates the allowance with:

```
require(allowance[from][msg.sender] >= value, "allowance too low");
```

With `value == 0` this is `0 >= 0`, which passes for any caller against any `from`. In a plain ERC20 that is harmless because nothing moves. Here `transferFrom()` then calls `token.sol#triggerUpdate()`, which drives the entire game state machine. The caller therefore gets unauthenticated write access to `userStats[from]` and `userStats[to]`, with no approval and no token balance.

Three concrete abuses, all reproduced:

Destroying accrued hold time. `transferFrom(victim, attacker, 0)` routes to `token.sol#updateUserStatsOnSell()`, setting `lastSellStamp = block.timestamp` and incrementing `sellStreak`. Because `getDaysHeld()` returns `0` whenever `lastSellStamp > lastBuyStamp`, a holder at day 89 is reset to day 0 by a single free call and stays there until they buy again - at which point the clock restarts from the new buy. Reward eligibility and the Sigil/Character traits are wiped:

Freezing a victim's tokens. 255 repetitions push `sellStreak` to `type(uint8).max`. The next increment panics, so every outgoing transfer by the victim - and by any spender they approved - reverts:

The mirror image, `transferFrom(uniswapV2Pair, victim, 0)`, routes to `updateUserStatsOnBuy()` and overflows `buyStreak` instead, so the pair can no longer send the victim tokens and their swaps revert:

Force-minting identities and pinning firstBuyAmount. `transferFrom(uniswapV2Pair, victim, 0)` on a victim with no history mints them an unsolicited, non-burnable NFT and records `firstBuyAmount = 0`. Since `updateUserStatsOnBuy()` only writes `firstBuyAmount` when `firstBuyStamp == 0`, the victim's real first purchase is never recorded and every growth-based trait becomes trivially maximal because the comparisons are against zero:

The consequence that sets the severity: the entire reward pot can be denied to every holder, permanently, for 23,905 gas per holder.

The reset is not temporary. Once `lastSellStamp` exceeds `lastBuyStamp`, `getDaysHeld` returns `0` and stays at `0` - only a new **buy** moves `lastBuyStamp` and the passage of time does nothing. So a single free call permanently pins a holder at zero days until they spend real money buying again, at which point the attacker repeats the call.

Applied across the holder set, this makes the 1,000,000-token reward pot - 10% of total supply - permanently unclaimable by anyone and destroys the protocol's central game mechanic. The cost asymmetry is 23,905 gas for the attacker against a 90-day wait plus a real token purchase for the victim. There is no profit motive required and no way for a victim to defend.

One more consequence worth calling out, because it makes the blast radius wider than it first looks: `uniswapV2Pair` is `address(0)` until `setUniswapV2Pair()` is called and `token.sol#triggerUpdate()` decides a buy with a bare `from == uniswapV2Pair`. Before launch, `address(0)` therefore *is* the pair and `transferFrom(address(0), victim, 0)` books a phantom pre-launch purchase. That mints NFTs with a `firstBuyStamp` earlier than the `launchTimeStamp` which `setUniswapV2Pair()` later writes - which then underflows in `traits.sol#computeBackgroundValue()` on a completely legitimate one-time launch call.

The last line is why the fix below closes this too - with `value > 0` enforced, the `address(0)` branch becomes unreachable, since nothing can ever hold an allowance from the zero address.

Unbounded supply and re-minting after a transfer are both intended - so every buyer mints an identity and someone who sells theirs and buys back in is meant to get a fresh one. What is *not* intended is that this vector lets addresses which never bought anything mint at will and lets it happen recursively inside a single transaction. `delete userStats[from]` re-arms the mint gate, `_safeMint` hands control to the recipient before `firstBuyStamp` is written and a zero-value call supplies the trigger

Recommendations

Reject zero-value transfers so the game state can only be moved by a real, authorised token movement:

```
function transferFrom(address from, address to, uint value) public returns
(bool) {
    require(value > 0, "zero value");
    require(balances[from] >= value, "balance too low");
    require(allowance[from][msg.sender] >= value, "allowance too low");
    ...
}
```

Apply the same guard in `transfer()`.

[H-01] Buying through an aggregator credits the router as the buyer, so the real purchaser never receives an identity

Severity

Impact:

Medium

Likelihood:

High

Description

`token.sol#triggerUpdate()` decides who bought by asking whether the *immediate* sender was the pair:

```
if (from == uniswapV2Pair) {
    updateUserStatsOnBuy(to, uint128(value));
}
```

That only holds when the pair pays the end user directly. A plain `UniswapV2Router` swap does exactly that and works correctly. But any route where an intermediary contract receives the output first - DEX

aggregators (1inch, 0x, CoW, Matcha), the Uniswap Universal Router when it takes a fee or unwraps, batching smart wallets - lands the tokens at the intermediary, which then forwards them.

In that shape the intermediary is credited with the entire buy: it gets `firstBuyStamp`, `firstBuyAmount` and the identity NFT. The forward on to the real purchaser is neither side of the pair, so it falls to the wallet-to-wallet branch, which sets only `lastTokenAmount`. The person who actually paid ends up with tokens and no identity at all.

Note that switching `_safeMint` to `_mint` does not fix this. That stops the revert for intermediaries which lack the receiver hook, but the NFT still lands on the router. The two outcomes are the two halves of the same routing gap: an intermediary either reverts the swap or absorbs the identity.

Because a later real buy finds `firstBuyStamp != 0` only for the router and not for the user, an aggregator buyer can still obtain an NFT by later buying directly from the pair - but their `firstBuyStamp` is then set to that later date, so they permanently lose the early-buyer Background and Character5 they paid for.

Recommendations

Do not infer the buyer from the immediate `from`. Have the buy path credit an explicitly identified recipient and treat pair-to-contract transfers as unclassified rather than as buys:

```
if (from == uniswapV2Pair) {
    // only credit EOAs; intermediaries forward on and get credited by
    their own leg
    if (to.code.length == 0) {
        updateUserStatsOnBuy(to, uint128(value));
    } else {
        userStats[to].lastTokenAmount = uint128(balances[to]);
    }
}
```

and credit the final recipient when an intermediary forwards, by treating a transfer *from* an address with no `firstBuyStamp` and a zeroed balance as a pass-through. A cleaner alternative is to drop pair-sniffing altogether and expose an explicit `registerIdentity()` the buyer calls themselves.

[H-02] Any ERC-721 operator approval is an unbounded, unrevocable-in-effect claim on the owner's entire token balance

Severity

Impact:

High

Likelihood:

Description

`nft.sol#transferFrom()` is authorised through OpenZeppelin, which permits the owner **or an approved address or an operator**. It then moves the owner's whole ERC20 position:

```
function transferFrom(address from, address to, uint256 tokenId) public
override {
    require(balanceOf(to) == 0, "recipient already owns a token");
    super.transferFrom(from, to, tokenId);           // OZ: owner OR
approved OR operator
    Token(contractOwner).transferByNFTContract(from, to);
}
```

and the ERC20 side performs no allowance check of its own - it cannot, because it only knows the NFT contract called it:

```
uint transferAmount = balanceOf(from);
if (transferAmount > 0 && balances[from] > 0) {
    balances[from] = 0;
    balances[to] += transferAmount;
}
```

The consequence is that ERC-721 approval semantics (permission over *one token*) and ERC-20 approval semantics (permission over *an amount*) are silently fused. `setApprovalForAll(marketplace, true)` - the routine, required step to list an NFT anywhere - hands that operator a standing right to move 100% of the owner's token balance to any address, with no sale, no payment and no escrow.

That the NFT carries the bag is the intended design and a *sale* moving the bag is intended too. What is not defensible is that the most common loss vector in NFTs - a phishing `setApprovalForAll` signature or a compromised marketplace operator - escalates here from "lose one NFT" to "lose the NFT plus every token you hold." Neither `approve` nor `setApprovalForAll` is overridden to bound or warn about that and nothing is emitted to make the coupling visible.

Recommendations

Break the fusion: require the owner to act, so delegated NFT movement cannot sweep the bag.

```
function transferFrom(address from, address to, uint256 tokenId) public
override {
    require(balanceOf(to) == 0, "recipient already owns a token");
    require(_msgSender() == from, "identity must be moved by its owner");
    super.transferFrom(from, to, tokenId);
    Token(contractOwner).transferByNFTContract(from, to);
    ...
}
```

If operator transfers must stay supported for marketplace listings, move only the NFT for a delegated call and require a separate, explicit opt-in from the owner before the balance follows.

[H-03] The value advertised to NFT buyers is stale by construction and cannot be corrected

Severity

Impact:

High

Likelihood:

Medium

Description

The identity NFT is freely transferable and `nft.sol` carries OpenSea metadata-refresh hooks throughout, so secondary-market trading is an expected use. The `ETH Amount` trait is the only on-chain figure the protocol publishes for valuing a position and it is computed inside `nft.sol#computeMetadata()`:

```
'{"display_type": "number", "trait_type": "ETH Amount"', ' "value": ',  
Strings.toString(Token(contractOwner).quoteTokensInWETH(balOfOwner))
```

from `balanceOf(owner)` at the moment the metadata is read. Three properties combine into a mispricing vector:

1. The figure is read at fetch time, not at settlement. `token.sol#transferByNFTContract()` moves whatever the seller holds when the transfer executes. The seller keeps control of that number until the last block before settlement.

2. Nothing tells the marketplace the number changed. The only refresh trigger is `nft.sol#triggerMetadataUpdate()`, which emits `MetadataUpdate(type(uint256).max)` - a *single-token* event naming a token that does not exist, where ERC-4906 requires `BatchMetadataUpdate` for a range. `supportsInterface` never advertises `0x49064906` either, so indexers gating on interface detection do not subscribe at all. A balance change therefore produces no refresh.

3. Once the seller exits, the figure can never be corrected. `quoteTokensInWETH(0)` reverts - confirmed against the live Base router - so `computeMetadata` reverts entirely for a zero-balance holder. Even a manual refresh cannot replace the cached number. It freezes at the last good reading, which is precisely the pre-dump figure.

`token.sol#transferByNFTContract()` migrates `userStats` to the new owner precisely so this **does not happen**. Without that migration the buyer's own history would drive the rendering and they

would receive a different picture from the one they bid on.

That intent is only partly achieved. `computeMetadata()` is built from nine fields and the migration covers seven of them:

Field	Source	Travels with the NFT
the six traits	<code>computeTraits(userStats, ...)</code>	yes - migrated
<code>birthday</code>	<code>getUserStats(owner).firstBuyStamp</code>	yes - migrated
Token Amount	<code>balanceOf(owner)</code>	no - read live
ETH Amount	<code>quoteTokensInWETH(balanceOf(owner))</code>	no - read live

The two that do not travel are precisely the two that price the asset. In the honest case this is invisible, because the balance moves with the NFT anyway. In the front-run case the buyer does not merely receive *different* metadata - they receive metadata that does not render at all, because the **ETH Amount** lookup reverts and takes the whole call with it. The scenario the migration was built to prevent therefore still occurs, in a strictly worse form and the broken refresh signal hides it by leaving the pre-sale version on display.

The sequence, using the numbers from the PoCs:

1. Alice holds 1,000,000 tokens and lists her identity. OpenSea caches an **ETH Amount** reflecting that position.
2. A buyer bids against that figure.
3. Before settlement lands, Alice sells her 1,000,000 tokens.
4. Settlement executes. `transferByNFTContract` moves **0 tokens**. The buyer receives an empty identity.
5. The listing's **ETH Amount** still shows the pre-dump value and now always will, because the metadata call reverts.

Alice keeps the sale proceeds and the dumped tokens. The buyer has no on-chain way to have checked, because the one number the protocol publishes for exactly this purpose is stale and unfixable.

Recommendations

The clean fix is to stop making the NFT's value a function of a number the seller controls after listing. Either snapshot the balance when the identity is listed or escrow it, so what settles is what was advertised.

If the coupling stays as designed, the minimum is to make the published figure honest rather than frozen:

```
// token.sol - never revert a display value
function quoteTokensInWETH(uint256 tokenAmount) external view returns
(uint256 wethAmount) {
    if (uniswapV2Pair == address(0) || tokenAmount == 0) return 0;
    ...
    try IUniswapV2Router02(UNISWAP_V2_ROUTER).getAmountsOut(tokenAmount,
path) returns (uint[] memory amounts) {
```

```

        wethAmount = amounts[1];
    } catch {
        wethAmount = 0;
    }
}

```

and emit a refresh signal indexers actually act on, so the number tracks the balance:

```

// nft.sol
function supportsInterface(bytes4 id) public view virtual override(ERC721,
IERC165) returns (bool) {
    return id == bytes4(0x49064906) || super.supportsInterface(id);
}

function triggerMetadataUpdate() public {
    if (lastMetadataUpdateTimestamp + 5 minutes <= block.timestamp) {
        lastMetadataUpdateTimestamp = block.timestamp;
        emit BatchMetadataUpdate(1, nextTokenId);
    }
}

```

Neither of those makes the value correct at settlement - only a snapshot or escrow does that - but together they stop a buyer being shown a number that is both wrong and permanently unrefreshable.

[M-01] `_safeMint` in the ERC20 transfer path makes buys revert for every contract recipient

Severity

Impact:

Medium

Likelihood:

Medium

Description

Every buy runs `token.sol#updateUserStatsOnBuy()`, which calls `nft.sol#mint()` on the recipient first purchase. `mint()` uses `_safeMint`, which invokes `onERC721Received` on the recipient and reverts if the recipient is a contract that does not implement it.

The recipient of a Uniswap V2 swap is whatever address the router was told to pay. That is routinely a contract: DEX aggregators (1inch, 0x, Matcha), smart-contract wallets, vaults and - for multi-hop routes - the next pair in the path. None of these implement `onERC721Received`. Each of them makes the pair `transfer()` revert, which reverts the whole swap.

The failure is permanent for a given contract, not transient: the mint is attempted on every buy until it succeeds, so such an address can never acquire the token through the pair at all.

There is a second consequence worth noting: because `_safeMint` hands control to an arbitrary recipient in the middle of an ERC20 transfer, this token behaves as a callback token for every integrator. That is the same class of hazard as ERC777 and it will make the token unsafe to integrate for protocols that assume ERC20 transfers are callback-free.

Recommendations

Use `_mint` instead of `_safeMint` and do not let identity minting be able to revert a token transfer:

```
function mint(address to) external returns (uint256 tokenId) {
    require(msg.sender == contractOwner, "only owner");
    if (balanceOf(to) == 0) {
        tokenId = ++nextTokenId;
        _mint(to, tokenId);
        userToTokenId[to] = tokenId;
    }
}
```

It is important to fix-interaction hazard and verify this before shipping. `_safeMint` is currently the only thing stopping an NFT from being minted to a contract that cannot handle one, including `Token` itself. I applied the fix to a copy and confirmed the new reachable state:

The first result is against the current source. The second is against a copy with `_safeMint` replaced by `_mint`, so it shows what the fix itself makes reachable rather than a defect in the code as it stands today.

A `pair` → `Token` transfer is classified as a buy, so the reward-pot contract is issued an identity NFT that nobody can ever move - `Token` has no code path to call `approve` or `transferFrom` on the NFT. I checked specifically whether this opens a route to the pot tokens: it does not, because `transferByNFTContract(address(token), attacker)` is only reachable through `nft.transferFrom`, which `_checkAuthorized` gates on an approval the Token contract can never grant. So this is a stuck NFT and a polluted `userStats[address(this)]`, not a theft. Still, skip the mint entirely for the protocol own addresses:

```
if (to != address(this) && to != contractOwner && balanceOf(to) == 0) { ...
}
```

[M-02] First buy after receiving tokens counts the pre-buy balance as extra leading to unearned WEAPON3 / CHARACTER3

Severity

Impact:

Medium

Likelihood:

Medium

Description

If an address receives R tokens before ever buying, then makes its first pair-buy of A : the pre-buy receive sets `lastTokenAmount = R` but leaves `firstBuyStamp == 0`, so on the first buy `updateUserStatsOnBuy` records `firstBuyAmount = A` (buy-only) and `lastTokenAmount = balances = R + A` (full balance). The growth traits compare `lastTokenAmount >= firstBuyAmount * 2`, i.e. $R + A \geq 2A$, which is true whenever $R \geq A$. So the address instantly earns WEAPON3 ("doubled initial buy") and CHARACTER3 ("100% balance growth since first buy") on its very first buy with zero organic growth — the intended "grew balance after entry" invariant is broken. This is because `firstBuyAmount` is the buy-only amount while `lastTokenAmount` is the full address balance including pre-existing tokens.

```
if (stats.firstBuyStamp == 0) {
    nftContract.mint(user);
    stats.firstBuyStamp = uint128(block.timestamp);
    stats.firstBuyAmount = amount; // buy-only amount A
    (ignores pre-buy balance R)
}
stats.lastTokenAmount = uint128(balances[user]); // full balance R + A
// traits.sol: lastTokenAmount >= firstBuyAmount * 2 ==> (R + A) >= 2A
==> true when R >= A -> WEAPON3 / CHARACTER3
```

Recommendations

On the first buy, base the growth reference on the post-buy balance rather than the buy amount.

[M-03] Missing EIP-4906 integration for auto-evolving metadata property

Severity

Impact:

Low

Likelihood:

High

Description

The "auto-evolving NFT" premise depends on marketplaces re-fetching each token's metadata when a holder's on-chain stats change. `triggerMetadataUpdate` which is called on every ERC20 transfer via `Token.triggerUpdate`, and on every NFT transfer tries to signal that with `emit MetadataUpdate(type(uint256).max)`.

But marketplaces rely on [EIP-4906](#) standard which declares two distinct events: `MetadataUpdate(uint256 _tokenId)` for a single token and `BatchMetadataUpdate(uint256 _fromTokenId, uint256 _toTokenId)` for a range/whole collection. A collection-wide refresh must be `BatchMetadataUpdate(0, type(uint256).max)` according to the standard. What is emitted is a single-token event pointing at token id $2^{256}-1$, which is never minted (ids run `1..nextTokenId`) — so indexers refresh nothing real, and every holder's displayed traits / Token Amount / ETH Amount go stale forever. Also, the contract declares its own local event `MetadataUpdate` instead of inheriting [IERC4906](#), and never advertises `supportsInterface(0x49064906)`, so indexers that work off of ERC-165 detection have no signal either.

```
event MetadataUpdate(uint256 _tokenId);           // local decl, not
IERC4906; no BatchMetadataUpdate

function triggerMetadataUpdate() public {
    if (lastMetadataUpdateTimestamp + 5 minutes > block.timestamp) {
        // ...
    } else {
        lastMetadataUpdateTimestamp = block.timestamp;
        emit MetadataUpdate(type(uint256).max);    // single-token event,
        nonexistent id
    }
}
// no supportsInterface override -> supportsInterface(0x49064906) == false
```

Recommendations

Inherit [IERC4906](#) (`import {IERC4906} from "@openzeppelin/contracts/interfaces/IERC4906.sol"`), drop the shadow event, and emit `BatchMetadataUpdate(0, type(uint256).max)` for a collection-wide refresh (or `MetadataUpdate(realTokenId)` targeted at the token whose stats changed). Override `supportsInterface` to also return true for `0x49064906`.

[M-04] `isCharacter2` anchors its 24-hour window to the global launch time instead of the user's first buy, making CHARACTER2 unreachable for anyone who buys after launch day

Severity

Impact:

Medium

Likelihood:

Medium

Description

`computeCharacterValue` has `userStats.firstBuyStamp` available but passes only `firstBuyAmount` into `isCharacter2`; the timing check uses `lastSellStamp - launchStamp` (time from the global launch to the sell), not time since the user's own first buy. The 50%-sell and full-exit clauses are "AND-ed" behind `timeSinceFirstBuy <= 24 hours / speedMultiplier`, so they only ever work for users that sold within 24h of the token's launch. A user who buys on day 40 and fully exits an hour later has `lastSellStamp - launchStamp ≈ 40 days`, fails the window, and falls through to CHARACTER1. The result: the CHARACTER2 ("Phantom" seller/exit archetype) branch is effectively dead for the entire post-launch-day population — the trait layer silently misclassifies its most common exit behavior.

```
function isCharacter2(uint128 firstBuyAmount, uint128 currentTokensHeld,
uint128 lastSellStamp, uint128 launchStamp) public view returns (bool) {
    if (launchStamp > lastSellStamp) return false;
    uint128 timeSinceFirstBuy = lastSellStamp - launchStamp;          //
time since LAUNCH, not since first buy
    if (timeSinceFirstBuy <= 24 hours / speedMultiplier &&
        (currentTokensHeld <= firstBuyAmount / 2 || currentTokensHeld ==
0)) {
        return true;
    }
    return false;
}
```

Recommendations

Pass `userStats.firstBuyStamp` into `isCharacter2` and compute `timeSinceFirstBuy = lastSellStamp - firstBuyStamp` (guarding `lastSellStamp >= firstBuyStamp`)

[M-05] Reward size is taken from the instantaneous balance and can be borrowed

Severity

Impact:

Medium

Likelihood:

Medium

Description

`token.sol#claimReward()` decides both eligibility and payout from `balances[msg.sender]` read at claim time:

```
uint128 tokensHeld = uint128(balances[msg.sender]);
require(traits.isCharacter6(daysHeld, tokensHeld, totalSupply), "...");
uint rewardAmount = balances[msg.sender];
```

The 90-day requirement is checked against `firstBuyStamp`, but the size requirement is checked against a balance that need not have been held for any length of time. Receiving tokens by plain transfer does not touch `lastSellStamp` or `firstBuyStamp`, so a qualifying account can be topped up immediately before the call and swept immediately after.

An account that genuinely held only the 0.1% minimum can therefore extract the full 0.25% maximum payout with borrowed capital:

Note also that `claimReward()` uses the live balance while `traits.sol#computeCharacterValue()` uses `lastTokenAmount` for the identical check, so the reward gate and the displayed Character trait can disagree.

Recommendations

Size the reward from the balance the protocol has actually been tracking and require it to be consistent with the live balance:

```
uint128 tokensHeld = stats.lastTokenAmount;
require(balances[msg.sender] >= tokensHeld, "balance decreased");
uint rewardAmount = tokensHeld;
```

[M-06] The rarest Weapon and Character traits have no absolute balance floor and cost 2 wei

Severity

Impact:

Low

Likelihood:

High

Description

Two of the six trait axes are scored against a baseline the user chooses, with no minimum.

`traits.sol#computeWeaponValue()` compares `lastTokenAmount` to `firstBuyAmount`, so the top tier is awarded for any relative doubling regardless of scale. Two 1 wei buys through the pair - ordinary behaviour, no bug required - give `firstBuyAmount = 1` and `lastTokenAmount = 2` and `2 >= 1 * 2` awards `WEAPON3`. Meanwhile a genuine 1,000,000-token buyer who never adds to their position receives `Weapon.NONE`.

`traits.sol#isCharacter5()` takes no balance argument at all:

```
function isCharacter5(uint128 firstBuyStamp, uint128 launchTimestamp,
uint128 lastSellStamp) public view returns (bool) {
    if (firstBuyStamp - launchTimestamp <= 72 hours / speedMultiplier &&
lastSellStamp == 0) {
        return true;
    }
}
```

Because `traits.sol#computeCharacterValue()` evaluates it before `isCharacter4()`, a 1 wei buyer from the launch window outranks a holder of 0.5% of supply, permanently and at no cost.

Note this is distinct from the degenerate case where `firstBuyAmount` is zero, which needs a separate authorisation defect to reach. This one needs nothing but a small first buy and the missing floor on `isCharacter5` is unrelated to it entirely.

The practical harm is to secondary buyers. The product's premise is that traits encode real conviction, so anyone paying a premium for a `WEAPON3` or `CHARACTER5` NFT is paying for a signal that costs a few wei to manufacture at scale.

Recommendations

Add an absolute floor alongside the relative test, expressed against supply so it scales with the deployment:

```
function computeWeaponValue(uint128 firstBuyAmount, uint128
lastTokenAmount, uint128 totalSupply)
    public pure returns (Weapon)
{
    if (lastTokenAmount * 100000 < totalSupply * 25) return Weapon.NONE; //
below 0.025%, no weapon
    ...
}
```

and give `isCharacter5()` the same minimum-holding requirement that `isCharacter4()` already has.

[M-07] The mint callback fires before `firstBuyStamp` is written, letting an attacker permanently brick any address's NFT for free

Severity

Impact:

Low

Likelihood:

High

Description

`token.sol#updateUserStatsOnBuy()` mints before it finishes writing state:

```
if (stats.firstBuyStamp == 0) {  
    nftContract.mint(user);  
    stats.firstBuyStamp = uint128(block.timestamp);  
    stats.firstBuyAmount = amount;  
}
```

`mint()` uses `_safeMint`, so the recipient receives control while it owns the NFT but still has `firstBuyStamp == 0`. From inside `onERC721Received` the recipient can call `nft.sol#transferFrom()` and hand the NFT - and, via `transferByNFTContract()`, its entire balance and the half-written stats - to a third address. The outer frame then finishes writing to the original address, which no longer holds the NFT.

The result is an NFT whose owner has `firstBuyStamp == 0`, which makes `traits.sol#computeBackgroundValue()` underflow on `firstBuyStamp - launchTimestamp`. Every metadata endpoint for that token reverts, permanently.

`nft.sol#mint()` has the same shape - it writes `userToTokenId[to]` after `_safeMint` returns, so the reentrant transfer's correction is clobbered by the outer frame. The result is a stale mapping entry pointing a non-owner at someone else's token, which `nft.sol#generateJSONMetadataFromAddress()` then resolves and renders instead of treating the caller as having no NFT:

This is not limited to self-inflicted damage. Chained with the vacuous allowance check on `token.sol#transferFrom() - require(allowance[from][msg.sender] >= value)` passes for any caller when `value` is zero - the attacker never needs to buy anything. They call `transferFrom(uniswapV2Pair, attackerContract, 0)` - free, no allowance, no tokens - and from inside their own `onERC721Received` they push the half-initialised NFT onto an arbitrary victim. The victim ends up holding a token whose metadata is permanently dead and because `nft.sol#transferFrom()` requires `balanceOf(to) == 0`, they can never be sent a working NFT to replace it. The contrast against a plain force-mint is the point: that one produces a functioning NFT, this one does not.

It also breaks the system's implicit invariant that every NFT owner has a recorded first buy.

Recommendations

Write the state before handing over control, in both places:

```
if (stats.firstBuyStamp == 0) {
    stats.firstBuyStamp = uint128(block.timestamp);
    stats.firstBuyAmount = amount;
    nftContract.mint(user);
}
```

and in `nft.sol#mint()`:

```
if (balanceOf(to) == 0) {
    tokenId = ++nextTokenId;
    userToTokenId[to] = tokenId;
    _mint(to, tokenId);
}
```

Combined with switching `_safeMint` to `_mint` there is no callback left at all, which closes both variants.

[M-08] The reward claim reverts instead of paying the remainder, permanently stranding the pot's tail

Severity

Impact:

Medium

Likelihood:

Medium

Description

`token.sol#claimReward()` computes the full entitlement first and then demands the pot cover it in one piece:

```
uint rewardAmount = balances[msg.sender];
uint maxReward = totalSupply / 400;
if (rewardAmount >= maxReward) { rewardAmount = maxReward; }
require(rewardPot >= rewardAmount, "Not enough rewards left in the pot");
```

There is no partial payout. Once the pot holds less than a claimant's entitlement, that claimant receives nothing rather than the remainder.

Because eligibility requires at least 0.1% of supply (10,000e18) and the entitlement is `min(balance, 25,000e18)`, every eligible claimant is owed at least 10,000e18. So the moment `rewardPot` drops below 10,000e18, no address in existence can satisfy the `require` - a larger balance only increases the entitlement and a smaller one fails `isCharacter6`. The residue is unreachable by construction and `Token` has no sweep function.

The user-facing loss is larger than the stranded dust: the denied claimant forfeits their entire entitlement, up to 25,000e18, not just the shortfall.

Recommendations

Pay what is available instead of reverting, which also drains the pot cleanly to zero:

```
if (rewardAmount > rewardPot) {
    rewardAmount = rewardPot;
}
require(rewardAmount > 0, "Reward pot is empty");
rewardPot -= rewardAmount;
```

[M-09] Providing liquidity is booked as a sell and withdrawing it mints an identity to someone who never bought

Severity

Impact:

Medium

Likelihood:

Medium

Description

The buy/sell classifier in `token.sol#triggerUpdate()` cannot distinguish a trade from a liquidity operation, because both touch the pair in the same direction.

`addLiquidity` makes the router pull the LP's tokens into the pair, so `to == uniswapV2Pair` and `token.sol#updateUserStatsOnSell()` fires against the LP. Providing liquidity therefore sets `lastSellStamp`, increments `sellStreak` and zeroes `buyStreak` - which zeroes `getDaysHeld`, destroys the Sigil, disqualifies Character4 and Character5 and stamps an Effect.

`removeLiquidity` makes the pair pay the LP, so `from == uniswapV2Pair` and `updateUserStatsOnBuy()` fires. Withdrawing your own liquidity mints an identity NFT to an address that never purchased anything, sets its `firstBuyStamp` and resets `sellStreak` to zero.

The protocol therefore punishes exactly the participants it most needs. An LP who supplies liquidity forfeits their accrued hold time and their path to the reward, permanently, unless they buy again and wait another 90 days.

This is a different code path and a different fix from the wallet-to-wallet branch of the same classifier, though both trace to the same two address comparisons.

Recommendations

Address-comparison alone cannot separate a swap from a liquidity operation - both are plain transfers to and from the pair. Either exclude a registered LP set from stat updates:

```
mapping(address => bool) public isLiquidityProvider; // owner-managed or
set at addLiquidity time

function triggerUpdate(address from, address to, uint value) internal {
    if (isLiquidityProvider[to] || isLiquidityProvider[from]) {
        userStats[from].lastTokenAmount = uint128(balances[from]);
        userStats[to].lastTokenAmount = uint128(balances[to]);
    } else if (from == uniswapV2Pair) {
        ...
    }
}
```

or accept that only genuine swaps can be classified and require buyers to register their identity explicitly, as suggested for the aggregator-routing case.

[M-10] An identity can be force-minted onto any address, blocking every NFT transfer to it

Severity

Impact:

Low

Likelihood:

High

Description

`nft.sol#transferFrom()` refuses any recipient that already holds a token:

```
require(balanceOf(to) == 0, "recipient already owns a token");
```

and `token.sol#updateUserStatsOnBuy()` mints an identity to whoever receives tokens from the pair, with no consent and no minimum amount. Anyone can therefore put an unwanted NFT on a target

address and, from that moment, every legitimate NFT transfer to that address reverts.

The obvious use is front-running a sale. A seller's settlement transaction is public before it lands; an attacker mints junk onto the buyer first and the transfer reverts:

The remedy is worse than it looks. To become able to receive an NFT again the victim must transfer the junk one away - and `transferByNFTContract` drags their entire ERC20 balance to wherever it goes:

A victim who sends the junk NFT to a wallet they control keeps the tokens, so this is forced migration rather than loss - which is why this is rated Low impact. But it is renewable indefinitely at roughly 24,000 gas per attempt against a victim who must migrate their whole position each time and it makes settlement on any venue unreliable.

This survives the obvious fix. Requiring `value > 0` in `token.sol#transferFrom()` removes the cost-free version, but a dust buy routed to the victim (`swapExactETHForTokens(..., to: victim, ...)`) force-mints just as well for the cost of a minimal swap.

Recommendations

Do not let an involuntary mint block a voluntary transfer. The cleanest fix is to make the identity mint opt-in rather than automatic:

```
// in updateUserStatsOnBuy: record the history, but only mint on request
if (stats.firstBuyStamp == 0) {
    stats.firstBuyStamp = uint128(block.timestamp);
    stats.firstBuyAmount = amount;
}
```

with a separate `claimIdentity()` the buyer calls for themselves. If automatic minting must stay, allow the recipient to burn or renounce an identity they did not ask for, so the block can be cleared without moving their balance.

[L-01] Every Border tier is 10× stricter than documented and contradicts the same percentage elsewhere in the file

Description

`traits.sol#computeBorderValue()` scales by `10000` but writes its tier constants in hundred-thousandths:

```
//Diamond = >= 0.5% of supply
if (tokensHeld * 10000 >= totalSupply * 500) {
    return Border.BORDER1;
}
```

`tokensHeld * 10000 >= totalSupply * 500` is `tokensHeld / totalSupply >= 500 / 10000`, which is 5% - ten times the 0.5% the comment specifies. The same error applies to all four tiers:

Comment	Actually implemented	Tokens needed at a 10,000,000 supply
BORDER1 "≥ 0.5%"	5%	500,000
BORDER4 "≥ 0.1%"	1%	100,000
BORDER2 "≥ 0.05%"	0.5%	50,000
BORDER3 "≥ 0.025%"	0.25%	25,000

What makes this unambiguous rather than a comment defect is that the identical percentages are implemented correctly elsewhere in the same contract. `traits.sol#isCharacter4()` expresses its documented 0.5% as `tokensHeld * 10000 >= totalSupply * 50` and `traits.sol#isCharacter6()` expresses its documented 0.1% as `* 10`. So "0.5% of supply" is written as both `* 50` and `* 500` in one file and only the Border version is wrong.

The consequence is that the Border axis is effectively dead for the protocol's own target user. `Border.NONE` now covers everything below 0.25% of supply, while the reward economy lives between 0.1% (`isCharacter6` eligibility) and 0.25% (`claimReward`'s `maxReward`). A holder who has maxed out every other axis still renders with no border.

Recommendations

Scale by `100000` so the existing constants mean what the comments say. Dividing the multipliers by ten instead does not work, because 0.025% is not expressible as a whole number over 10,000.

```
if (tokensHeld * 100000 >= totalSupply * 500) { return Border.BORDER1; }  
// 0.5%  
if (tokensHeld * 100000 >= totalSupply * 100) { return Border.BORDER4; }  
// 0.1%  
if (tokensHeld * 100000 >= totalSupply * 50) { return Border.BORDER2; }  
// 0.05%  
if (tokensHeld * 100000 >= totalSupply * 25) { return Border.BORDER3; }  
// 0.025%  
return Border.NONE;
```

The upper bounds in the existing `&&` clauses are unreachable once the tiers are checked in descending order and can be dropped. At `uint128`, `tokensHeld * 100000` peaks at `1e30` against a `3.4e38` ceiling, so there is no overflow risk.

[L-02] NFT transfer overwrites the recipient's own UserStats

Description

`token.sol#transferByNFTContract()` assigns the sender whole struct over the recipient:

```
userStats[to] = userStats[from];
```

`nft.sol#transferFrom()` guarantees the recipient owns no NFT, but it does not guarantee they own no tokens - anyone can hold the ERC20 without ever having bought from the pair. When such a holder receives an identity NFT, their own history is discarded and `lastTokenAmount` is replaced by the sender, which now understates their real balance. Every percentage-of-supply trait (`computeBorderValue`, `isCharacter4`, `isCharacter6` via `computeCharacterValue`) reads `lastTokenAmount`, so the recipient's traits are computed against the wrong figure until their next transfer refreshes it.

Recommendations

Refresh the derived field after the migration so it always reflects reality:

```
userStats[to] = userStats[from];
userStats[to].lastTokenAmount = uint128(balances[to]);
delete userStats[from];
```

[L-03] `setServerBaseURI` is gated on `tx.origin`

Description

`nft.sol` records `deployer = tx.origin` in its constructor and uses it as the sole authorisation for `nft.sol#setServerBaseURI()`. `Token` meanwhile records `owner = msg.sender`. The two diverge whenever the token is not deployed directly by an EOA - which is the expected case for a launchpad template, where a factory calls `new Token(...)`.

With a factory deployment, `owner` is the factory and `deployer` is the EOA. With a Safe or an ERC-4337 account, `tx.origin` is the executing signer or the bundler - neither of which is the intended controller. `tx.origin` authorisation is also incompatible with any account-abstraction flow.

Separately, `serverBaseURI` is already set in the constructor and `setServerBaseURI` is one-shot, so there is exactly one chance to correct a bad URL before the metadata endpoint is frozen forever.

Recommendations

Authorise against an explicitly passed owner rather than `tx.origin`:

```
constructor(string memory name_, string memory symbol_, string memory
baseURL, address _traitC, address _admin)
    ERC721(name_, symbol_)
{
    contractOwner = msg.sender;
    traitContract = _traitC;
```

```
serverBaseURI = baseUrl;
deployer = _admin;
}
```

[L-04] `speedMultiplier` tightens some traits while loosening others and the Sigil ignores it entirely

Description

The header comment states that `speedMultiplier` accelerates trait evolution, but the same divisor is applied to two dimensionally different quantities, with opposite effects.

Where it divides an eligibility window - `traits.sol#computeBackgroundValue()` (`72 hours / speedMultiplier, 15/30/60 days / speedMultiplier`), `traits.sol#isCharacter5()` and `isCharacter2()` - a larger multiplier shrinks the window and makes the desirable tier harder to earn.

Where it divides a holding requirement - `traits.sol#isCharacter6()` (`daysHeld >= 90 / speedMultiplier`) - a larger multiplier shrinks the bar and makes the tier easier.

`traits.sol#computeSigilValue()` reads `speedMultiplier` nowhere at all; its 7/30/60/90-day thresholds are fixed, so it is the only time-based trait that does not accelerate. That contradicts the documented intent directly - the file header says "Speed multiplier is used to accelerate the evolution of traits" and the Sigil is one of the six traits, evolving purely with time.

Identical user behaviour on two deployments differing only in the multiplier:

A deployer who sets `speedMultiplier = 3` intending to speed the game up opens the reward gate three times sooner while making the genesis Background and Character5 strictly harder to obtain and leaves the Sigil unchanged.

Recommendations

Decide which direction "faster" means and apply it consistently. If it should compress the whole timeline, divide holding requirements and leave eligibility windows alone - a launch window is an absolute property of the launch, not a duration the player accrues:

```
// windows stay fixed
if (timeSinceFirstBuy <= 72 hours) { return Background.BACKGROUND3; }
```

and add the divisor to `computeSigilValue()` thresholds so every accrual-based trait scales together.

[L-05] The Effect trait reads a streak rather than a sell count and is erased by a 1 wei buy

Description

`traits.sol#computeEffectValue()` carries two comments, neither of which matches the code:

```
function computeEffectValue(uint sellStreak) public pure returns (Effect) {  
    //Effect is a user that has sold more than 50% of their initial buy  
    amount
```

and at the top of the file, "Effect is based on how many times the user has sold". The input is `sellStreak`, which is a consecutive-sell counter, not a lifetime count - `token.sol#updateUserStatsOnBuy()` resets it to 0 on any buy of any size. No sell amount is compared against `firstBuyAmount` anywhere in the codebase, so the 50% threshold in the comment is not implemented at all.

The result is that the Effect trait is trivially resettable. A user with five real sells into the pair carries `EFFECT2`; buying 1 wei back returns them to `Effect.NONE` and erases the record.

The same reset also clears the Effect for a user who is simply accumulating normally, so the trait does not measure what either comment describes.

Recommendations

Track what the comment claims. Add a monotonic lifetime counter that buys do not reset and drive the trait from it:

```
// in UserStats  
uint16 sellCount;  
  
// in updateUserStatsOnSell, alongside the existing streak  
if (stats.sellCount < type(uint16).max) { stats.sellCount += 1; }
```

then pass `stats.sellCount` into `computeEffectValue()`. If the intent really is the 50%-of-initial-buy rule, compare `lastSellAmount` against `firstBuyAmount` instead and drop the counter entirely - but pick one, because the two comments describe different mechanics.

[L-06] Splitting a position across wallets multiplies the reward payout

Description

`token.sol#claimReward()` bounds the payout per address:

```
uint rewardAmount = balances[msg.sender];  
uint maxReward = totalSupply / 400;    // 0.25% of supply  
if (rewardAmount >= maxReward) { rewardAmount = maxReward; }
```

and records the claim per address, with the comment "Each user can claim once!". But eligibility via `traits.sol#isCharacter6()` only requires 0.1% of supply, while the cap sits at 0.25%. The gap between those two numbers is the whole problem: the cap binds a wallet, not a person and the entry ticket is cheaper than the cap.

A holder therefore earns strictly more by splitting the same capital across wallets. Every wallet is a genuine buyer with real 90-day history - no bug from any other finding is needed and the position never leaves the holder control.

The second test is one actor holding 10% of supply, deployed as 100 wallets sitting exactly on the eligibility floor. Every one of them buys from the pair, waits the full 90 days and claims legitimately. The pot is emptied by a single participant and every other holder in the protocol is left with nothing.

This depends on no other defect. Every wallet involved is a genuine buyer with genuine 90-day history and a genuine balance.

The claim is keyed per address and the pot is first-come-first-served by construction, both of which read as deliberate. Rated low on that basis and reported for the record rather than as a defect.

One part of it was not addressed and is worth a decision, because the fix is free: the eligibility floor (0.1%) sits below the payout cap (0.25%). That inversion is what makes splitting strictly more profitable than not splitting, rather than merely possible. Setting the floor equal to the cap makes splitting payout-neutral at no cost to honest holders.

Recommendations

The per-address cap cannot be enforced on-chain against an actor who controls many addresses. Make the payout proportional rather than capped-per-wallet, so splitting yields nothing:

```
// pay a fixed fraction of the pot proportional to the holding, with no
per-wallet floor advantage
uint rewardAmount = (rewardPot * tokensHeld) / totalSupply;
```

If the flat cap must stay, raise the eligibility threshold to meet it (require the claimant to hold at least `maxReward`, i.e. 0.25%, not 0.1%) so that splitting is at best payout-neutral instead of strictly better.

[L-07] Reward pot is not funded at deployment

Description

The contract initializes `rewardPot` to 10 percent of the total supply but assigns the full supply to the deployer. The Token contract itself therefore has no balance to pay the rewards it advertises.

```
uint256 public rewardPot = totalSupply / 10;
```

```
balances[msg.sender] = totalSupply;  
normalTransfer(address(this), msg.sender, rewardAmount);
```

An otherwise eligible user can pass all reward checks and still have `claimReward` revert because the payment source has a zero balance. The reward program remains unavailable until an operator performs an undocumented manual transfer to the Token contract.

Recommendations

- Allocate the reward reserve to the Token contract during construction.
- Alternatively, provide an explicit funding function that updates the recorded reward pot and the available token balance atomically.

[L-08] Metadata quotes use an unvalidated fixed router

Description

After any nonzero pair is configured, JSON metadata generation calls a hard-coded Base router and assumes a direct Token-to-WETH route. The configured pair is only used as a nonzero switch and is not validated against the router, chain, factory, or quote path.

```
if (uniswapV2Pair == address(0)) return 0;  
  
address[] memory path = new address[](2);  
path[0] = address(this);  
path[1] = WETH;  
uint256[] memory amounts = IUniswapV2Router02(UNISWAP_V2_ROUTER)  
    .getAmountsOut(tokenAmount, path);  
wethAmount = amounts[1];
```

If the router is unavailable, the deployment is on another chain, or the direct route does not exist, the quote call reverts and all JSON metadata reads fail. When it succeeds, the displayed price can still be unreliable because the route is not tied to the configured pair and can be manipulated through a low-liquidity pool.

Recommendations

- Configure and validate the router, wrapped native token, pair, and chain at deployment.

[L-09] Similar transaction fails due to overflow after a user's 255 consecutive action in the same direction

Description

`stats.buyStreak += 1` and `stats.sellStreak += 1` operate on `uint8` fields under solidity ^0.8.20 checked arithmetic. A streak is reset only by an opposite-direction action, so 256 consecutive same-direction actions (buys with no intervening sell, or vice-versa) overflow the counter on the 256th, and since this executes inside `triggerUpdate`, the whole token transfer reverts.

```
struct UserStats {
    uint128 firstBuyStamp;
    uint128 lastSellStamp;
    uint128 lastBuyStamp;
    uint128 lastBuyAmount;
    uint128 lastSellAmount;
    uint128 firstBuyAmount;
    uint128 lastTokenAmount;
    uint8 sellStreak;
    uint8 buyStreak;
}
```

```
stats.buyStreak += 1;
...
stats.sellStreak += 1;
```

Recommendations

Use larger uint types

[L-10] A sold-out NFT holder (ERC20 balance 0) reverts on-chain metadata via `quoteTokensInWETH(0)`

Description

`computeMetadata` reads `balOfOwner = Token.balanceOf(tokenOwner)` and passes it straight into `Token.quoteTokensInWETH(balOfOwner)` for the "ETH Amount" attribute. `quoteTokensInWETH` only guards `uniswapV2Pair == address(0)`; with the pair set (the normal live state) and `tokenAmount == 0` it calls `getAmountsOut(0, path)`, which reverts on Uniswap V2 with `INSUFFICIENT_INPUT_AMOUNT`. So the entire metadata call reverts for any NFT holder who has sold down to a zero token balance — an potentially common end-state (buy → get NFT → sell everything → keep the NFT). `generateJSONMetadataFromTokenId` / `generateJSONMetadataFromAddress` revert for them until they re-acquire a nonzero balance.

```
function quoteTokensInWETH(uint256 tokenAmount) external view returns
(uint256 wethAmount) {
    if (uniswapV2Pair == address(0)) return 0;          // guards unset pair,
    NOT tokenAmount == 0
    address[] memory path = new address[](2);
```

```

    path[0] = address(this);
    path[1] = WETH;
    uint[] memory amounts =
    IUniswapV2Router02(UNISWAP_V2_ROUTER).getAmountsOut(tokenAmount, path); //
    getAmountsOut(0) -> revert
    wethAmount = amounts[1];
}

```

Recommendations

Add a 0 amount check like in `quoteNFTHolderTokensInWETH`

[L-11] Re-calling `setUniswapV2Pair` resets `launchTimeStamp`, causing an underflow revert that bricks metadata for all prior buyers

Description

`setUniswapV2Pair` unconditionally sets `launchTimeStamp = block.timestamp` on every call and has no one-time guard. If the owner calls it a second time (e.g. to migrate the pair) after users have bought, `launchTimeStamp` becomes greater than existing `firstBuyStamp` values. Then `computeBackgroundValue` and `isCharacter5` compute `firstBuyStamp - launchTimeStamp` on `uint128`, which underflows and reverts under checked arithmetic. This permanently bricks `generateNFTTraits`, `generateJSONMetadataFromTokenId`, and `generateJSONMetadataFromAddress` for every pre-reset holder.

```

// token.sol
function setUniswapV2Pair(address pair) public {
    require(msg.sender == owner, "...");
    uniswapV2Pair = pair;
    launchTimeStamp = block.timestamp;           //reset every call
}
// traits.sol
uint128 timeSinceFirstBuy = firstBuyStamp - launchTimeStamp; //underflow
if firstBuyStamp < launchTimeStamp

```

Recommendations

Don't reset `launchTimeStamp` in the setter (set it once in the constructor), or guard the setter to run once.

[L-12] `getDaysHeld` off-by-one: a same-block buy+sell (`lastSellStamp == lastBuyStamp`) is

ignored, inflating days-held

Description

`getDaysHeld`'s "has sold" branch uses `if (lastSellStamp > lastBuyStamp) return 0;` — a strict `>`. When a buy and a sell land in the same block (`updateUserStatsOnBuy` sets `lastBuyStamp = block.timestamp`, `updateUserStatsOnSell` sets `lastSellStamp = block.timestamp`, so they're exactly equal) and nothing happens afterward, the equality falls through to the else branch and keeps accruing days from `lastBuyStamp` as if the user never sold. The intended semantics ("sold at/after the last buy $\Rightarrow$ 0 days held") require `>=`. Downstream, as the fake clock crosses 7/30/60/90 days, `computeSigilValue` awards SIGIL1–4 and — for a same-block partial sell that leaves $\geq 0.1\%$ of supply — `isCharacter6` (and thus the `claimReward` gate) mislabels the account CHARACTER6 and reward-eligible, despite it having dumped in the same block. Distinct from the (rejected) legitimate re-buy case (`lastBuyStamp > lastSellStamp`, where the clock should restart) and from #4 (the never-sold `lastSellStamp == 0` branch); this is a separate exact-equality off-by-one in the has-sold branch.

```
if (lastSellStamp > lastBuyStamp) {                // strict > misses
    lastSellStamp == lastBuyStamp
    return 0;
} else {
    return fromDurationToDays(uint128(block.timestamp) -
    uint128(lastBuyStamp)); // credits days despite a same-block sell
}
```

Recommendations

Change `if (lastSellStamp >= lastBuyStamp) { return 0; }` so a sell at the same timestamp as the last buy zeroes the days-held credit.

[I-01] "ETH Amount" metadata attribute is emitted in raw wei while "Token Amount" is human-scaled

Description

In `computeMetadata`, "Token Amount" is rendered as `Strings.toString(balOfOwner / 1e18)` (whole tokens), but "ETH Amount" is `Strings.toString(quoteTokensInWETH(balOfOwner))`, not divided by `1e18`. `quoteTokensInWETH` returns WETH in wei (18 decimals, `amounts[1]` from the router). Both attributes use `display_type: "number"`, which marketplaces render literally, so a holder worth 0.5 ETH shows "ETH Amount": `500000000000000000` next to a small "Token Amount" — the two numeric traits are on scales that differ by $\sim 1e18$.

```
'{"display_type": "number", "trait_type": "Token Amount", "value": ',
Strings.toString(balOfOwner / 1e18),                // human units
...'
```

```
'{"display_type": "number", "trait_type": "ETH Amount"', ' "value": ',  
Strings.toString(Token(contractOwner).quoteTokensInWETH(balOfOwner)),    //  
raw wei
```

Recommendations

Scale the values to match each other